

CI32/CY32 -DBS

UNIT 5

Introduction to Transaction Processing

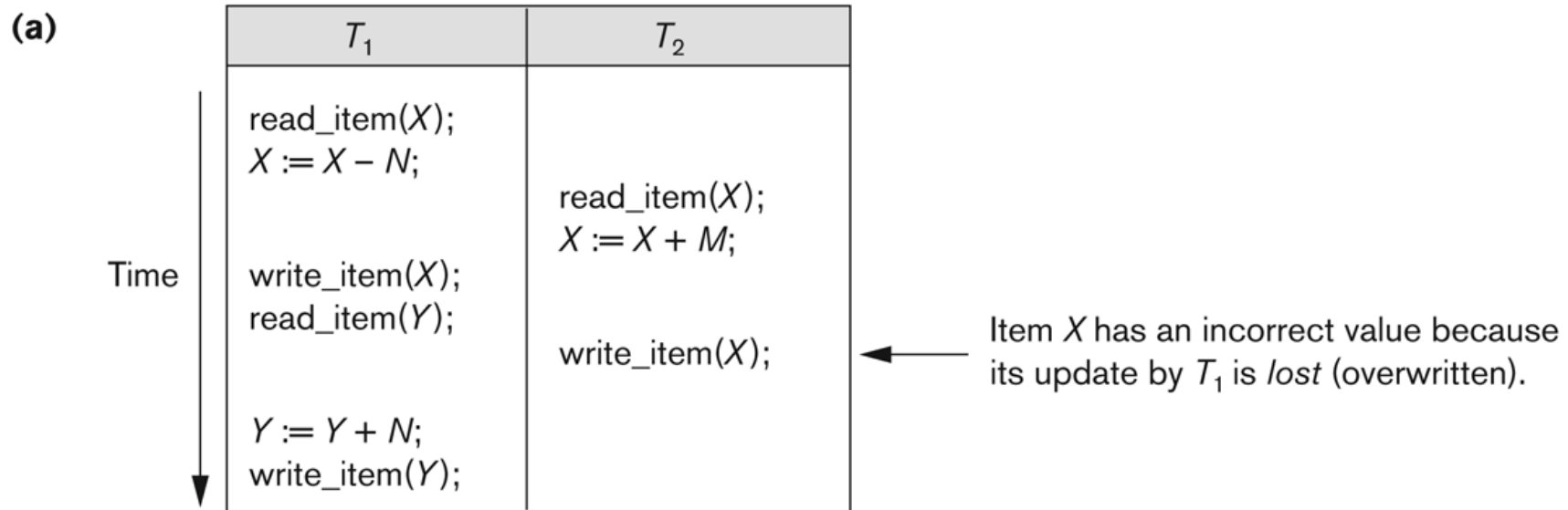
Why Concurrency Control is needed:

- **The Lost Update Problem**
 - This occurs when two transactions that access the same database items have their operations interleaved in a way that makes the value of some database item incorrect.
- **The Temporary Update (or Dirty Read) Problem**
 - This occurs when one transaction updates a database item and then the transaction fails for some reason (see Section 17.1.4).
 - The updated item is accessed by another transaction before it is changed back to its original value.
- **The Incorrect Summary Problem**
 - If one transaction is calculating an aggregate summary function on a number of records while other transactions are updating some of these records, the aggregate function may calculate some values before they are updated and others after they are updated.

Concurrent execution is uncontrolled: (a) The lost update problem.

Figure 17.3

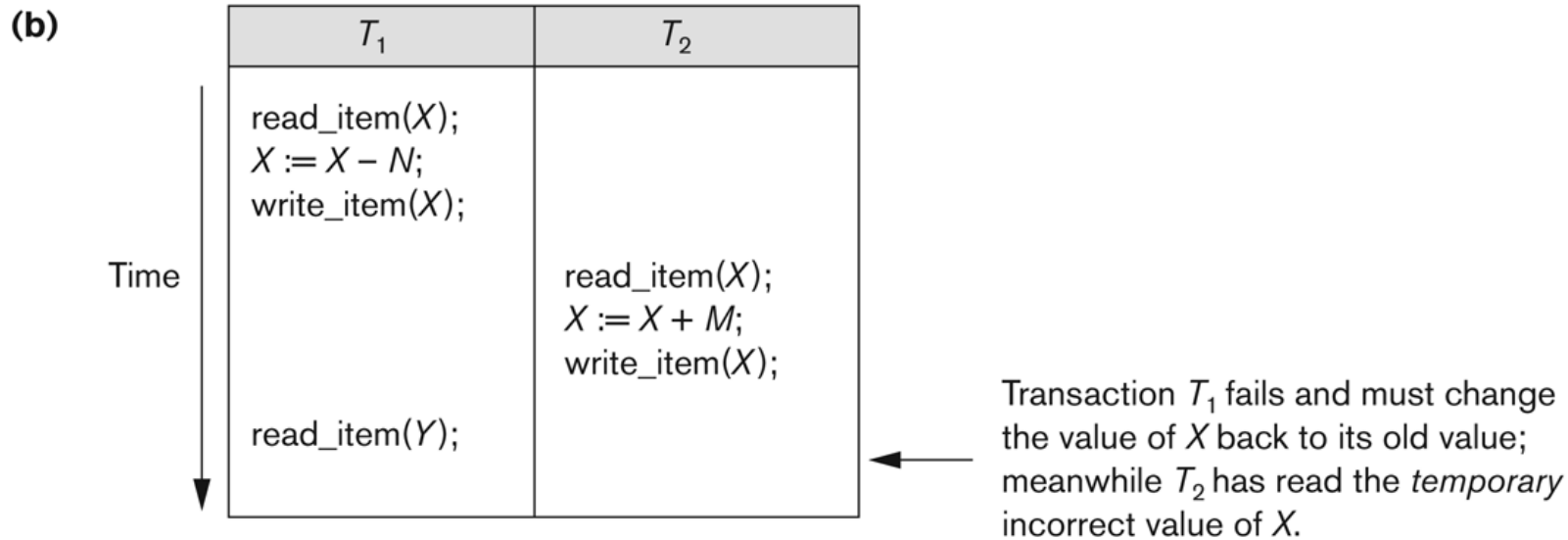
Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.



Concurrent execution is uncontrolled: (b) The temporary update problem.

Figure 17.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.



Concurrent execution is uncontrolled: (c) The incorrect summary problem.

Figure 17.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

(c)

T_1	T_3
<pre>read_item(X); X := X - N; write_item(X); read_item(Y); Y := Y + N; write_item(Y);</pre>	<pre>sum := 0; read_item(A); sum := sum + A; ⋮ read_item(X); sum := sum + X; read_item(Y); sum := sum + Y;</pre>

← T_3 reads X after N is subtracted and reads Y before N is added; a wrong summary is the result (off by N).

Introduction to Transaction Processing

Why **recovery** is needed:

(What causes a Transaction to fail)

1. A computer failure (system crash):

A **hardware or software error** occurs in the computer system during transaction execution. If the hardware crashes, the contents of the computer's internal memory may be lost.

2. A transaction or system error:

Some operation in the transaction may cause it to fail, such as **integer overflow or division by zero**. Transaction failure may also occur because of **erroneous parameter values or because of a logical programming error**. In addition, the **user may interrupt** the transaction during its execution.

Introduction to Transaction Processing

Why **recovery** is needed (Contd.):

(What causes a Transaction to fail)

3. Local errors or exception conditions detected by the transaction:

Certain conditions necessitate cancellation of the transaction. For example, data for the transaction may not be found. A condition, such as **insufficient account balance in a banking database**, may cause a transaction, such as a fund withdrawal from that account, to be canceled.

A programmed abort in the transaction causes it to fail.

4. Concurrency control enforcement:

The concurrency control method may decide to abort the transaction, to be restarted later, because it **violates serializability** or because several transactions are in a state of **deadlock** .

Introduction to Transaction Processing

Why **recovery** is needed (contd.):

(What causes a Transaction to fail)

5. Disk failure:

Some disk blocks may lose their data because of a **read or write malfunction** or because of **a disk read/write head crash**. This may happen during a read or a write operation of the transaction.

6. Physical problems and catastrophes:

This refers to an endless list of problems that includes **power or air-conditioning failure, fire, theft**, sabotage, overwriting disks or tapes by mistake, and mounting of a wrong tape by the operator.

Transaction and System Concepts

- A **transaction** is an atomic unit of work that is either completed in its entirety or not done at all.
 - For recovery purposes, the system needs to keep track of when the transaction starts, terminates, and commits or aborts.
- **Transaction states:**
 - Active state
 - Partially committed state
 - Committed state
 - Failed state
 - Terminated State

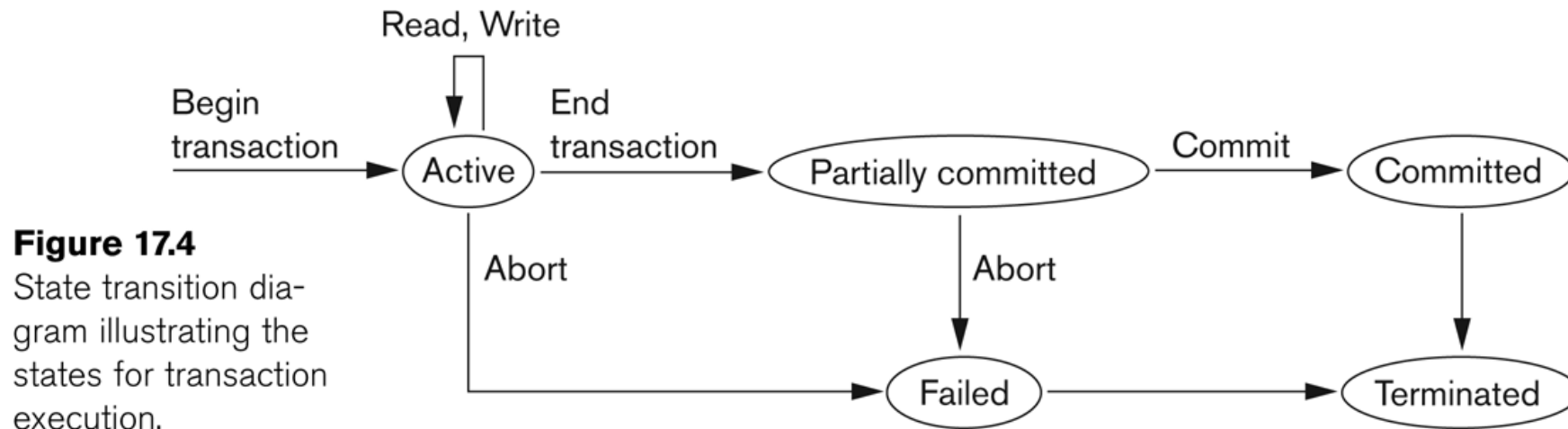
Transaction and System Concepts

- Recovery manager keeps track of the following operations:
 - **begin_transaction**: This marks the beginning of transaction execution.
 - **read** or **write**: These specify read or write operations on the database items that are executed as part of a transaction.
 - **end_transaction**: This specifies that read and write transaction operations have ended and marks the end limit of transaction execution.
 - At this point it may be necessary to check whether the changes introduced by the transaction can be permanently applied to the database or whether the transaction has to be aborted because it violates concurrency control or for some other reason.

Transaction and System Concepts

- Recovery manager keeps track of the following operations (cont):
 - **commit_transaction**: This signals a successful end of the transaction so that any changes (updates) executed by the transaction can be safely committed to the database and will not be undone.
 - **rollback** (or **abort**): This signals that the transaction has ended unsuccessfully, so that any changes or effects that the transaction may have applied to the database must be undone.

State transition diagram illustrating the states for transaction execution



Transaction and System Concepts

- The System Log
 - **Log or Journal:** The log keeps **track of all transaction operations that affect the values of database items.**
 - This information may be needed to permit recovery from transaction failures.
 - The **log is kept on disk**, so it is **not affected by any type of failure except for disk or catastrophic failure.**
 - In addition, the log is periodically backed up to archival storage (tape) to guard against such catastrophic failures.

Transaction and System Concepts

- The System Log (cont):
 - T in the following discussion refers to a unique **transaction-id** that is generated automatically by the system and is used to identify each transaction:
 - Types of log record:
 - [start_transaction,T]: Records that transaction T has started execution.
 - [write_item,T,X,old_value,new_value]: Records that transaction T has changed the value of database item X from old_value to new_value.
 - [read_item,T,X]: Records that transaction T has read the value of database item X.
 - [commit,T]: Records that transaction T has completed successfully, and affirms that its effect can be committed (recorded permanently) to the database.
 - [abort,T]: Records that transaction T has been aborted.

Characterizing Schedules based on Recoverability

- **Transaction schedule or history:**
 - When transactions are executing **concurrently in an interleaved fashion, the order of execution of operations** from the various transactions forms what is known as a transaction schedule (or history).
- A **schedule (or history) S** of n transactions $T_1, T_2, \dots, T_n$:
 - It is an **ordering of the operations** of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i .
 - Note, however, that operations from other transactions T_j can be interleaved with the operations of T_i in S .

Characterizing Schedules based on Recoverability

Schedules classified on recoverability:

- **Recoverable schedule:**
 - One where no transaction needs to be rolled back.
 - A schedule S is recoverable **if no transaction T in S commits** until all transactions T' that have written an item that T reads have committed.
- **Cascadeless schedule:**
 - One where **every transaction reads only the items that are written by committed transactions.**

Characterizing Schedules based on Recoverability

Schedules classified on recoverability (contd.):

- **Schedules requiring cascaded rollback:**
 - A schedule in which **uncommitted transactions that read an item from a failed transaction** must be rolled back.
- **Strict Schedules:**
 - A schedule in which a **transaction can neither read or write an item X until the last transaction that wrote X has committed.**

Characterizing Schedules based on Serializability

- Serial schedule:
 - A schedule S is serial if, for every transaction T participating in the schedule, all the operations of T are executed consecutively in the schedule.
 - Otherwise, the schedule is called nonserial schedule.
- Serializable schedule:
 - A schedule S is serializable if it is equivalent to some serial schedule of the same n transactions.

Characterizing Schedules based on Serializability

- Result equivalent:
 - Two schedules are called result equivalent if they produce the same final state of the database.
- Conflict equivalent:
 - Two schedules are said to be conflict equivalent if the order of any two conflicting operations is the same in both schedules.
- Conflict serializable:
 - A schedule S is said to be conflict serializable if it is conflict equivalent to some serial schedule S' .

Characterizing Schedules based on Recoverability

- **Transaction schedule or history:**
 - When transactions are executing **concurrently in an interleaved fashion, the order of execution of operations** from the various transactions forms what is known as a transaction schedule (or history).
- A **schedule (or history)** S of n transactions $T_1, T_2, \dots, T_n$:
 - It is an **ordering of the operations** of the transactions subject to the constraint that, for each transaction T_i that participates in S , the operations of T_i in S must appear in the same order in which they occur in T_i .
 - Note, however, that operations from other transactions T_j can be interleaved with the operations of T_i in S .

Isolation levels in Transaction Processing

- A transaction isolation level is defined by the following phenomena –
- **Dirty Read** – A Dirty read is the situation when a **transaction reads a data that has not yet been committed**.
- **Non Repeatable read** – Non Repeatable read occurs when a **transaction reads same row twice, and get a different value each time**.
- **Phantom Read** – Phantom Read occurs **when two same queries are executed, but the rows retrieved by the two, are different**.

Based on these phenomena, The SQL standard defines four isolation levels :

Isolation levels in Transaction Processing

- **Read Uncommitted** – Read Uncommitted is the lowest isolation level. In this level, one transaction may read not yet committed changes made by other transaction, thereby allowing dirty reads. In this level, transactions are not isolated from each other.
- **Read Committed** – This isolation level guarantees that **any data read is committed at the moment it is read**. Thus it does not allow dirty read. The transaction **holds a read or write lock on the current row**, and thus prevent other transactions from reading, updating or deleting it.

Isolation levels in Transaction Processing

- **Repeatable Read** – This is the most restrictive isolation level. The transaction holds **read locks on all rows it references and writes locks on all rows it inserts, updates, or deletes**. Since other transaction cannot read, update or delete these rows, consequently it avoids non-repeatable read.
- **Serializable** – This is the Highest isolation level. **A *serializable* execution is guaranteed to be serializable.** Serializable execution is defined to be an execution of operations in which concurrently executing transactions appears to be serially executing.

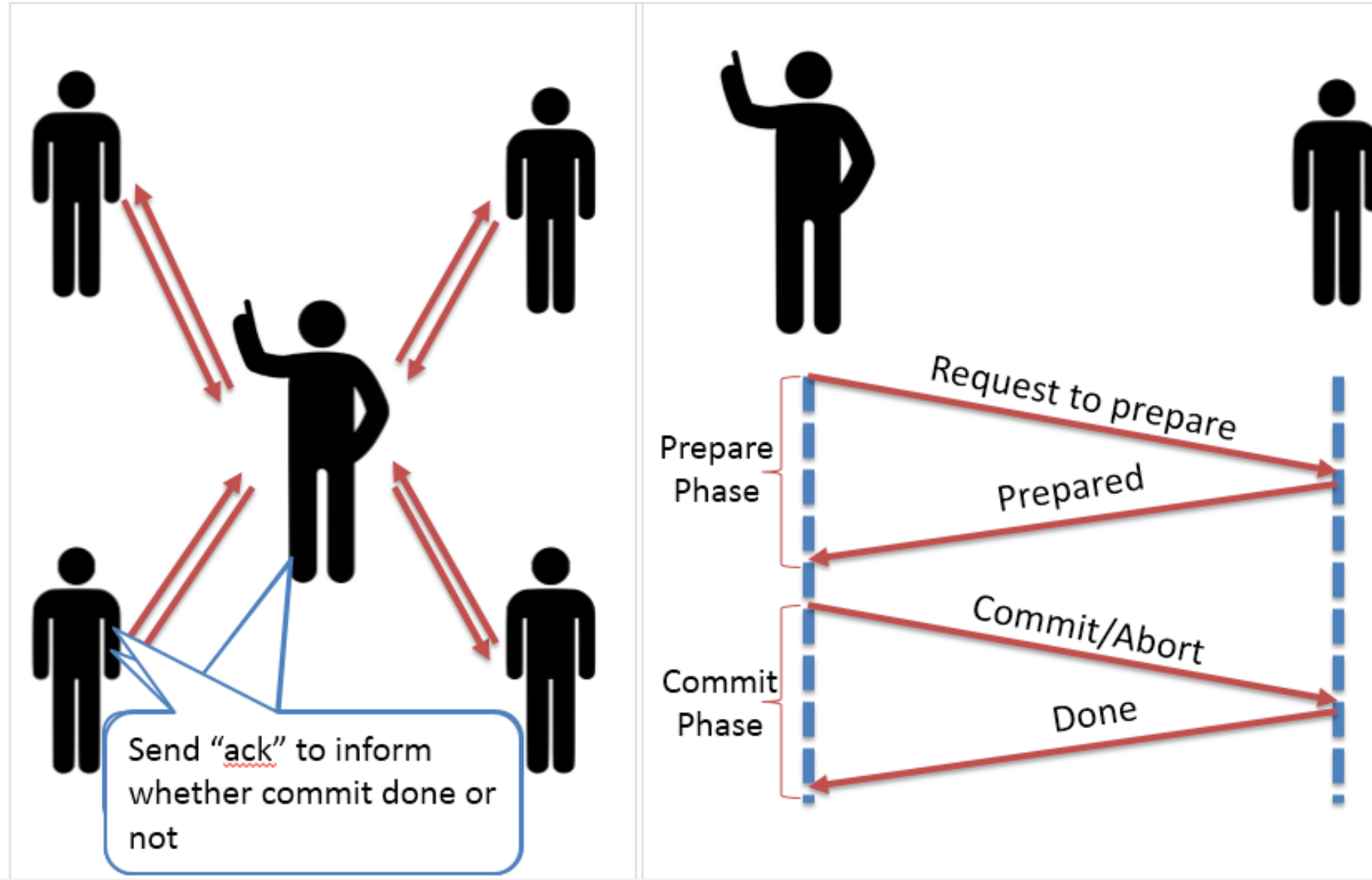
Two phase commit protocol

- Two phase commit protocol **ensures that all participants perform the same action** (either to commit or to rollback a transaction).
- It is designed to ensure that either all the databases are updated or none of them, so that the databases remain synchronized.
- In two phase commit protocol there is one node which is act as a coordinator or controlling site and all other participating node are known as cohorts or participant or slave.
- Coordinator (controlling site) – the component that coordinates with all the participants.
- Cohorts (Participants/Slaves) – each individual node except coordinator are participant.

Two phase commit protocol

- As the name suggests, the two phase commit protocol involves two phases.
 1. Commit request phase OR Prepare phase
 2. Commit/Abort phase

Two phase commit protocol



Two phase commit protocol

1. Commit Request Phase (Obtaining Decision)

- **After each slave has locally completed its transaction**, it **sends a “DONE” message** to the controlling site.
- When the **controlling site has received “DONE” message** from all slaves, it **sends a “Prepare” (prepare to commit) message** to the slaves.
- The **slaves vote** on whether they still **want to commit or not**.
- **If a slave wants to commit**, it **sends a “Ready” message**.
- A **slave that does not want to commit sends a “Not Ready” message**.
- This may happen when the slave has conflicting concurrent transactions or there is a timeout.

Two phase commit protocol

2. Commit Phase (Performing Decision)

- 1) After the **controlling site has received "Ready" message** from all the slaves:
 - The controlling site **sends a "Global Commit" message** to the slaves.
 - The **slaves commit** the transaction and **send a "Commit ACK" message** to the controlling site.
 - When the **controlling site receives "Commit ACK" message from all the slaves**, it considers the **transaction as committed**.

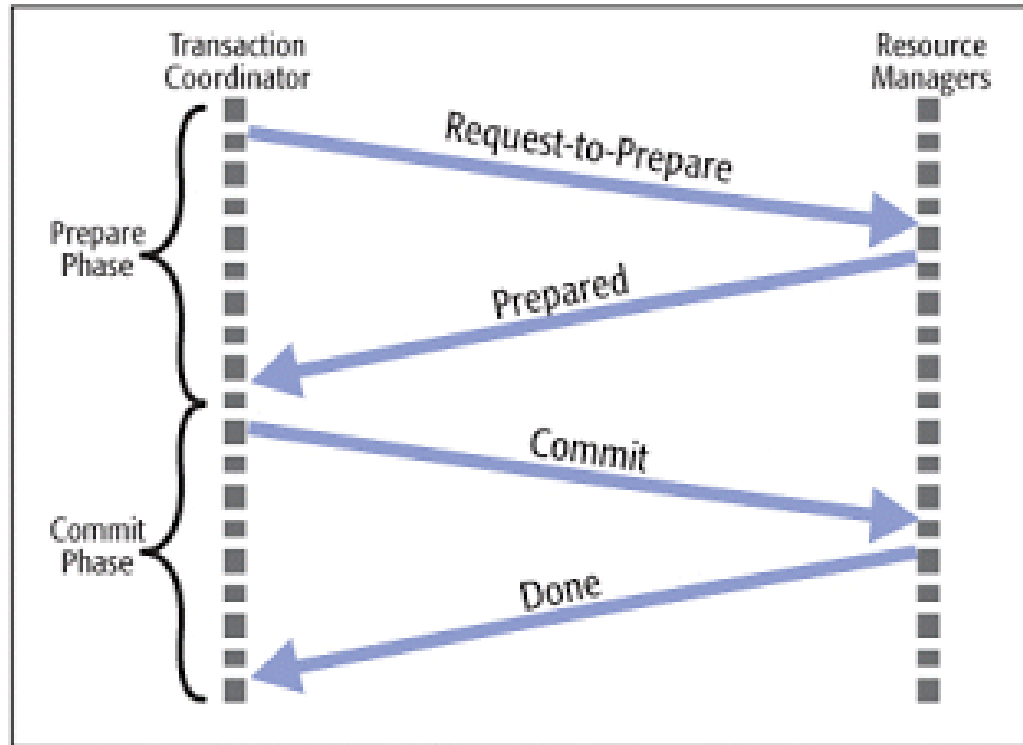
Two phase commit protocol

2. Commit Phase (Performing Decision)

- 2) After the **controlling site has received the first “Not Ready” message** from any slave:
 - The **controlling site sends a “Global Abort” message** to the slaves.
 - The **slaves abort** the transaction and **send a “Abort ACK” message** to the controlling site.
 - When the **controlling site receives “Abort ACK” message from all the slaves**, it considers the **transaction as aborted**.

Two phase commit protocol

- Atomic commitment protocol
- Distributed algorithm that coordinates all process and participate in a distributed atomic transaction, whether to commit or abort T



Limitation

- Block Problem:
 - Blocking reduces the availability of system since blocked transactions keep all the resources until they receive final command from coordinator.

Database recovery

- There are many situations in which a **transaction may not reach a commit or abort point.**
 - **Operating system crash**
 - **DBMS crash**
 - **System might lose power (power failure)**
 - **Disk may fail** or other **hardware may fail (disk/hardware failure)**
 - **Human error**
- In any of above situations, data in the database may become inconsistent or lost.

Database recovery

- For example, if a transaction has completed 30 out of 40 write instructions to the database when the DBMS crashes, then the database may be in an inconsistent state as only part of the transaction's work was completed.
- Database recovery is the **process of restoring the database and the data to a consistent state**.
- This may include **restoring lost data up to the point of the event** (e.g. system crash).